Top 50 Machine Learning Questions — Deep Answers

Fundamentals

1. Bias-variance tradeoff

Total expected error decomposes as:

$$E[(y - \hat{f}(x))^2] = \underbrace{(\text{Bias}[\hat{f}])^2}_{\text{underfit}} + \underbrace{\text{Var}[\hat{f}]}_{\text{overfit}} + \underbrace{\sigma^2}_{\text{irreducible}}$$

- **Bias** = error from wrong assumptions (model too simple). High bias → underfit.
- **Variance** = error from sensitivity to training data (model too complex). High variance → overfit.
- Increasing model complexity lowers bias but raises variance. The sweet spot minimizes total error. Regularization, more data, and ensembling shift this balance.

2. Overfitting vs underfitting

- **Overfit**: low train error, high validation error. Model memorized noise.
- **Underfit**: high train AND validation error. Model too weak.
- **Detect**: plot learning curves (train vs val error over epochs / data size). Diverging curves = overfit; both high and flat = underfit.
- **Fix overfit**: regularization, dropout, more data, simpler model, early stopping. **Fix underfit**: more features, more complex model, train longer.

3. Supervised / unsupervised / semi-supervised / RL

- **Supervised**: labeled data, learn `x → y` (classification, regression).
- **Unsupervised**: no labels, find structure (clustering, PCA, autoencoders).
- **Semi-supervised**: small labeled + large unlabeled set (pseudo-labeling, consistency regularization).
- **RL**: agent takes actions in an environment, learns from reward signal to maximize cumulative return. No fixed dataset — data comes from interaction.

4. Curse of dimensionality

As dimensions grow, volume grows exponentially, so data becomes sparse. Consequences:

- Distances become meaningless — nearest and farthest points converge in ratio.
- Need exponentially more samples to cover the space.
- Overfitting risk rises. **Fixes**: dimensionality reduction (PCA, t-SNE, UMAP), feature selection, regularization.

5. Generative vs discriminative

- **Discriminative**: model `P(y|x)` directly (logistic regression, SVM, neural nets). Learns the decision boundary.

- **Generative**: model `P(x|y)` and `P(y)`, then use Bayes to get `P(y|x)` (Naive Bayes, GMM, GANs, VAEs). Can generate new samples.
- Discriminative usually wins on pure prediction; generative wins when you need to sample or handle missing data.

6. Parametric vs non-parametric

- **Parametric**: fixed number of parameters regardless of data size (linear/logistic regression). Fast, strong assumptions.
- **Non-parametric**: parameters grow with data (k-NN, decision trees, kernel SVM). Flexible, need more data, slower.

7. Regularization — L1 vs L2

Add penalty to loss to shrink weights and reduce overfitting.

- **L2 (Ridge)**: $\lambda \Sigma w_i^2$. Shrinks weights smoothly toward zero, never exactly zero. Handles correlated features well.
- **L1 (Lasso)**: $\lambda \Sigma |w_i|$. Drives some weights to exactly zero → feature selection.
- **Elastic Net**: combines both.

```python
loss = mse(y, y_hat) + lam * np.sum(np.abs(w))      # L1
loss = mse(y, y_hat) + lam * np.sum(w**2)           # L2
```

L1's corner geometry (diamond constraint) makes it hit axes → sparsity.

8. Cross-validation

Estimate generalization by rotating which data is held out.

- **k-fold**: split into k parts, train on k−1, validate on 1, repeat k times, average.
- **Stratified**: preserves class ratios per fold (essential for imbalance).
- **LOOCV**: k = n, low bias, high variance, expensive.
- **Time-series**: use forward-chaining (no future leakage).

9. Train/validation/test — why three

- **Train**: fit parameters.
- **Validation**: tune hyperparameters / select model.
- **Test**: final unbiased estimate, touched once. If you tune on the test set, you leak information and overstate performance.

10. Data leakage

When information unavailable at prediction time sneaks into training.

- **Target leakage**: a feature is a proxy for the label (e.g., "payment_received" when predicting default).

- **Train-test contamination**: scaling/imputing before the split. **Prevent**: fit all transforms on train only (use `Pipeline`), audit features for future info, split before any stats.

Algorithms

11. Linear regression

`ŷ = Xβ`, minimize MSE. Closed form:

$$\hat{\beta} = (X^T X)^{-1} X^T y$$

Assumptions: linearity, independence of errors, homoscedasticity (constant error variance), normally distributed errors, no perfect multicollinearity.

12. Logistic regression

Models `P(y=1|x) = σ(wᵀx)` where `σ(z) = 1/(1+e⁻ᶻ)`.

- **Why sigmoid**: maps $\mathbb{R} \to (0,1)$, gives a valid probability; its log-odds are linear in x.
- **Why log-loss**: MSE on sigmoid is non-convex; cross-entropy is convex and is the negative log-likelihood of the Bernoulli model.

$$L = -\frac{1}{n} \sum [y \log \hat{p} + (1-y) \log(1-\hat{p})]$$

13. Decision tree splits

Greedily pick the feature/threshold that maximizes purity gain.

- **Gini**: `1 − Σpᵢ²` (faster, default in CART).
- **Entropy**: `−Σpᵢ log pᵢ`; information gain = parent entropy − weighted child entropy. Both measure impurity; results are usually similar. Splitting continues until stopping criteria (max depth, min samples).

14. Random forest vs gradient boosting

- **RF**: bagging — many deep trees trained **in parallel** on bootstrapped samples + feature subsets, averaged. Reduces **variance**. Robust, hard to overfit.
- **GBM**: boosting — trees built **sequentially**, each fitting the residual/gradient of the prior ensemble. Reduces **bias**. More accurate but sensitive to tuning and overfitting.

15. XGBoost improvements over plain GBM

- Regularized objective (L1 + L2 on leaf weights).
- Second-order Taylor expansion (uses gradient **and** Hessian).
- Sparsity-aware split finding (handles missing values natively).
- Weighted quantile sketch for approximate splits.
- Parallelized column blocks, cache-aware, shrinkage + column subsampling.

16. SVM

Finds the hyperplane maximizing the margin (distance to nearest points = support vectors).

$$\min \tfrac{1}{2}\|w\|^2 + C\sum \xi_i \quad \text{s.t.} \quad y_i(w^T x_i + b) \geq 1 - \xi_i$$

- **C**: soft-margin tradeoff (small C = wider margin, more slack).
- **Kernel trick**: replace $x_i^T x_j$ with $K(x_i, x_j)$ to work in high-dim space without computing it (RBF, polynomial). RBF: $\exp(-\gamma\|x-x'\|^2)$.

17. k-NN

Lazy learner — no training. Predict by majority vote (or mean) of k nearest neighbors by distance (Euclidean, cosine).

- Small k → high variance (noisy); large k → high bias (oversmoothed).
- Pick k via cross-validation, often odd to break ties. Scale features first; suffers from curse of dimensionality.

18. k-means

Partition into k clusters minimizing within-cluster sum of squares.

1. Init k centroids (k-means++ for good seeds).
2. Assign each point to nearest centroid.
3. Recompute centroids as cluster means.
4. Repeat until stable. **Pick k**: elbow method (inertia vs k), silhouette score, gap statistic. Assumes spherical, equal-size clusters.

19. PCA

Find orthogonal directions of maximum variance.

1. Center data.
2. Compute covariance matrix $C = (1/n)X^TX$.
3. Eigen-decompose (or SVD): eigenvectors = principal components, eigenvalues = variance explained.
4. Project onto top-k components. Keep components explaining ~95% cumulative variance. Linear, unsupervised.

20. Naive Bayes

Applies Bayes' theorem with the "naive" assumption that features are conditionally independent given the class:

$$P(y|x) \propto P(y)\prod_i P(x_i|y)$$

"Naive" because features are rarely truly independent — yet it works well (esp. text/spam). Fast, needs little data. Use Laplace smoothing for unseen features.

Evaluation

21. Precision vs recall vs F1

- **Precision** = TP/(TP+FP): of predicted positives, how many correct. Use when false positives are costly (spam filter).
- **Recall** = TP/(TP+FN): of actual positives, how many caught. Use when false negatives are costly (cancer screening).
- **F1** = harmonic mean `2PR/(P+R)`: balances both, good for imbalance.

22. ROC-AUC vs PR-AUC

- **ROC**: TPR vs FPR across thresholds; AUC = P(random positive ranked above random negative). Threshold-independent.
- **PR-AUC**: precision vs recall. **Better for heavy imbalance** because ROC can look optimistic when negatives dominate (FPR barely moves).

23. Confusion matrix

	Pred Pos	**Pred Neg**
Actual Pos	TP	FN
Actual Neg	FP	TN

Everything derives from it: accuracy `(TP+TN)/all`, precision, recall, specificity `TN/(TN+FP)`.

24. Class imbalance

- **Data**: oversample minority (SMOTE), undersample majority, or both.
- **Algorithm**: class weights, focal loss.
- **Threshold**: move decision threshold using PR curve.
- **Metrics**: use F1/PR-AUC/balanced accuracy, never raw accuracy.

25. Regression metrics

- **RMSE**: `√(mean((y−ŷ)²))` — penalizes large errors, same units as y, sensitive to outliers.
- **MAE**: `mean(|y−ŷ|)` — robust to outliers, linear.
- **R^2**: fraction of variance explained, `1 − SSres/SStot`. Use MAE if outliers shouldn't dominate; RMSE if large errors are especially bad.

26. Good baseline

Simplest reasonable predictor: majority class (classification) or mean/median (regression), or a plain logistic/linear model. Everything complex must beat it — otherwise complexity isn't justified.

27. Type I vs Type II errors

- **Type I (FP)**: predict positive when actually negative — false alarm.
- **Type II (FN)**: predict negative when actually positive — miss. The precision/recall tradeoff is exactly the FP/FN tradeoff.

Feature Engineering & Data

28. Missing values

- **Delete**: rows/cols if few and random (MCAR).
- **Impute**: mean/median/mode, model-based (KNN, MICE), or forward-fill (time series).
- **Flag**: add "is_missing" indicator — missingness itself can be signal.
- Tree models (XGBoost) handle NaNs natively.

29. Categorical encoding

- **One-hot**: low cardinality; no ordinal assumption but explodes dimensions.
- **Label/ordinal**: only if a true order exists.
- **Target/mean encoding**: replace category with mean target — powerful but leaks; use CV-folds/smoothing.
- **Embeddings**: high cardinality in neural nets; learns dense vectors.

30. Feature scaling

Needed for distance- and gradient-based methods (k-NN, SVM, k-means, neural nets, PCA, regularized linear). **Not** needed for trees.

- **Standardization**: `(x−μ)/σ` — assumes roughly Gaussian.
- **Min-max**: `(x−min)/(max−min)` → [0,1].
- **Robust**: uses median/IQR — outlier-resistant.

31. Outliers

- **Detect**: z-score, IQR (`< Q1−1.5·IQR` or `> Q3+1.5·IQR`), isolation forest, DBSCAN.
- **Handle**: remove (if error), cap/winsorize, transform (log), or use robust models/metrics. Don't blindly delete — outliers can be the signal (fraud).

32. Feature selection

- **Filter**: correlation, chi-square, mutual information (model-agnostic, fast).
- **Wrapper**: recursive feature elimination, forward/backward (uses model, slow).
- **Embedded**: L1/Lasso, tree feature importance (built into training).

33. High-cardinality features

Target/frequency encoding, hashing trick, embeddings, or group rare categories into "other". Avoid one-hot (dimension blowup).

Optimization

34. Gradient descent variants

`θ ← θ − η∇L(θ)`

- **Batch**: full dataset per step — stable, slow, memory-heavy.
- **Stochastic (SGD)**: one sample — noisy, fast, escapes local minima.
- **Mini-batch**: 32–512 samples — best of both, hardware-friendly. Default.

35. Learning rate

Step size η.

- Too high → diverges/oscillates.
- Too low → slow, may stall. Use schedules (step decay, cosine), warmup, or adaptive optimizers. Find via LR-range test.

36. Local minima & saddle points

- **Local minimum**: lower than neighbors but not global.
- **Saddle point**: gradient zero but a min in one direction, max in another — far more common in high-dim than true local minima and the real bottleneck. Momentum/Adam help escape both.

37. Loss vs cost function

- **Loss**: error for a single example.
- **Cost/objective**: aggregate over the dataset (mean loss + regularization). Often used interchangeably, but that's the technical distinction.

38. Momentum / Adam

- **Momentum**: `v ← βv + ∇L; θ ← θ − ηv`. Accumulates velocity, smooths oscillations, accelerates.
- **Adam**: combines momentum (1st moment) + RMSProp (2nd moment) with bias correction. Per-parameter adaptive learning rates. Robust default.

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t, \quad v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2$$

Deep Learning

39. Backpropagation

Efficient computation of loss gradients w.r.t. all weights via the chain rule, propagating error backward layer by layer. Forward pass computes activations; backward pass computes `∂L/∂w` reusing cached intermediates. Enables gradient descent in deep nets.

40. Vanishing / exploding gradients

In deep nets, repeated multiplication of small (<1) gradients → vanish; large (>1) → explode. **Fixes**: ReLU activations, proper init (He/Xavier), batch norm, residual/skip connections, gradient clipping (explosion), LSTM/GRU gates (RNNs).

41. Activation functions

- **Sigmoid**: (0,1), saturates → vanishing gradients, not zero-centered.
- **Tanh**: (−1,1), zero-centered, still saturates.
- **ReLU**: `max(0,x)` — no saturation for positives, fast, but "dying ReLU". Variants: LeakyReLU, GELU, ELU. Default hidden: ReLU/GELU. Output: sigmoid (binary), softmax (multiclass), linear (regression).

42. Dropout & batch norm

- **Dropout**: randomly zero activations (prob p) during training → prevents co-adaptation, acts as ensemble. Off at inference (scaled).
- **Batch norm**: normalize layer inputs per mini-batch (zero mean, unit var) + learnable scale/shift. Speeds training, adds slight regularization, allows higher LR.

43. CNN vs RNN vs Transformer

- **CNN**: local spatial patterns via convolution + weight sharing. Images, grids.
- **RNN/LSTM**: sequential, maintains hidden state. Time series, older NLP. Struggles with long dependencies, no parallelism.
- **Transformer**: self-attention, fully parallel, captures long-range dependencies. Dominant for NLP and increasingly vision.

44. Attention / self-attention

Attention lets each element weight the relevance of others.

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

- **Self-attention**: Q, K, V all come from the same sequence — every token attends to every other, learning context. Multi-head runs several in parallel to capture different relation types.

45. Transfer learning

Reuse a model pretrained on a large dataset, then fine-tune on your smaller task. Early layers learn general features (edges, syntax); you adapt the head/later layers. Saves data and compute; standard for CV (ImageNet) and NLP (BERT/LLMs).

Practical / MLOps

46. Model drift

- **Data drift**: input distribution shifts. **Concept drift**: `P(y|x)` relationship changes.
- **Detect**: monitor input stats (PSI, KL divergence), prediction distribution, and live metrics.
- **Fix**: scheduled/triggered retraining, online learning, alerting, champion-challenger setups.

47. Why ensembles help

- **Bagging** (RF): averages high-variance, low-bias models → cuts variance (errors decorrelate).
- **Boosting** (GBM): sequentially reduces bias.
- **Stacking**: a meta-model learns to combine base models. Works when base learners are accurate **and** diverse — their errors partly cancel.

48. Great offline, poor in prod — debug

Check, in order: train/serve skew (different preprocessing), data leakage inflating offline scores, distribution shift, feature availability/latency at serve time, label delay, and the offline metric not matching the business objective. Add logging + shadow deployment to isolate.

49. SHAP vs LIME

Both explain individual predictions.

- **LIME**: fits a local linear surrogate around one point. Fast, approximate, can be unstable.
- **SHAP**: Shapley values from game theory — fair feature attribution with consistency guarantees. Slower but theoretically grounded; gives global + local views.

50. When is a model "good enough" to ship

It clears the bar defined **before** training: beats the baseline and current production model, meets the business metric threshold (not just ML metric), performs acceptably across key segments (fairness), meets latency/cost/interpretability constraints, and is robust on a held-out recent slice. "Good enough" = the marginal cost of more accuracy exceeds its value.